

# lab3-4 基于 UDP 服务设计可靠传输协议并编程实现

---

才子桓 2212700 密码科学与技术

## 实验要求

---

**实验3-1：** 利用数据报套接字在用户空间实现面向连接的可靠数据传输，功能包括：建立连接、差错检测、确认重传等。流量控制采用停等机制，完成给定测试文件的传输。

**实验3-2：** 在实验 3-1 的基础上，将停等机制改成基于滑动窗口的流量控制机制，采用固定窗口大小，支持累积确认，完成给定测试文件的传输。

**实验3-3：** 在实验 3-2 的基础上，选择实现一种拥塞控制算法，也可以是改进的算法，完成给定测试文件的传输。

**实验3-4：** 基于给定的实验测试环境，通过改变网路的延迟时间和丢包率，完成下面3组性能对比实验：

- (1) 停等机制与滑动窗口机制性能对比；
- (2) 滑动窗口机制中不同窗口大小对性能的影响；
- (3) 有拥塞控制和无拥塞控制的性能比较。

要求如下：

1. 实现单向传输。
2. 对于每一个任务要求给出详细的协议设计。
3. 给出实现的拥塞控制算法的原理说明。
4. 完成给定测试文件的传输，显示传输时间和平均吞吐率。
5. 性能测试指标包括吞吐率、文件传输时间等，给出图形结果并进行分析。
6. 完成详细的实验报告（每个任务完成一份）。
7. 编写的程序应该结构清晰，具有较好的可读性。
8. 提交程序源码、可执行文件和实验报告。

## 实验三总结

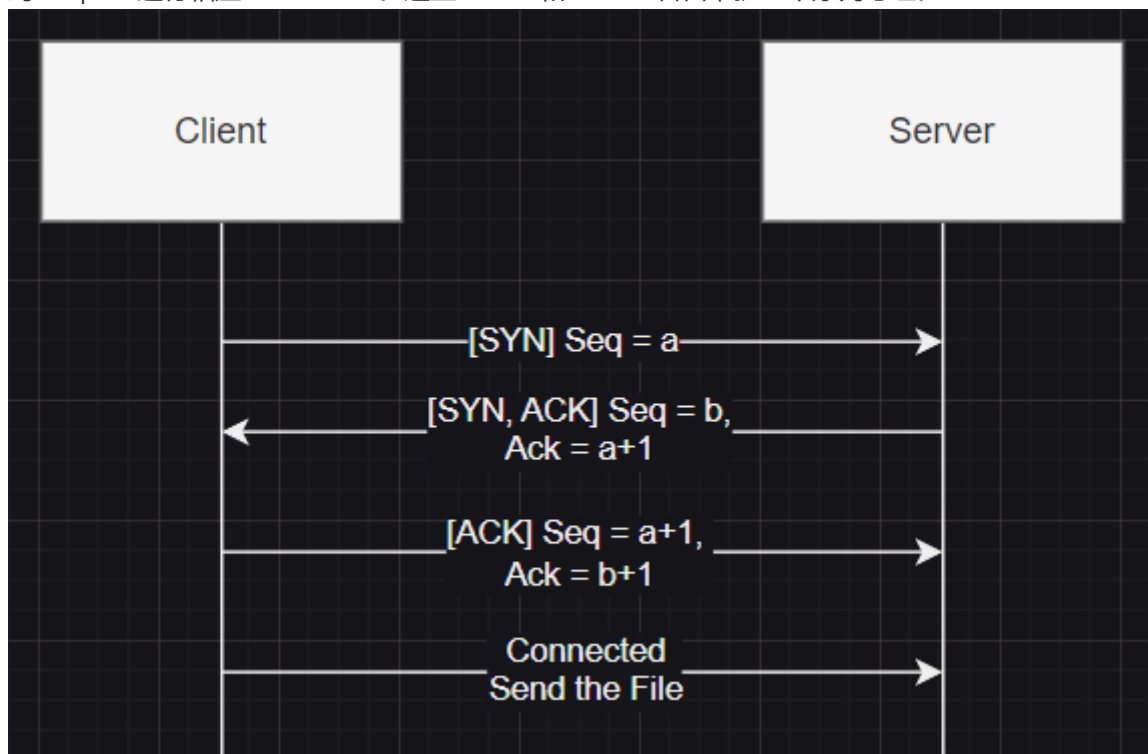
---

本次实验基于 UDP 协议设计可靠传输协议。在本次实验中，我仿照 TCP 可靠传输协议，设计一套可靠传输方案。

这里我对实验 3-1、3-2、3-3 的协议设计进行了总结，由于篇幅问题不展示代码。

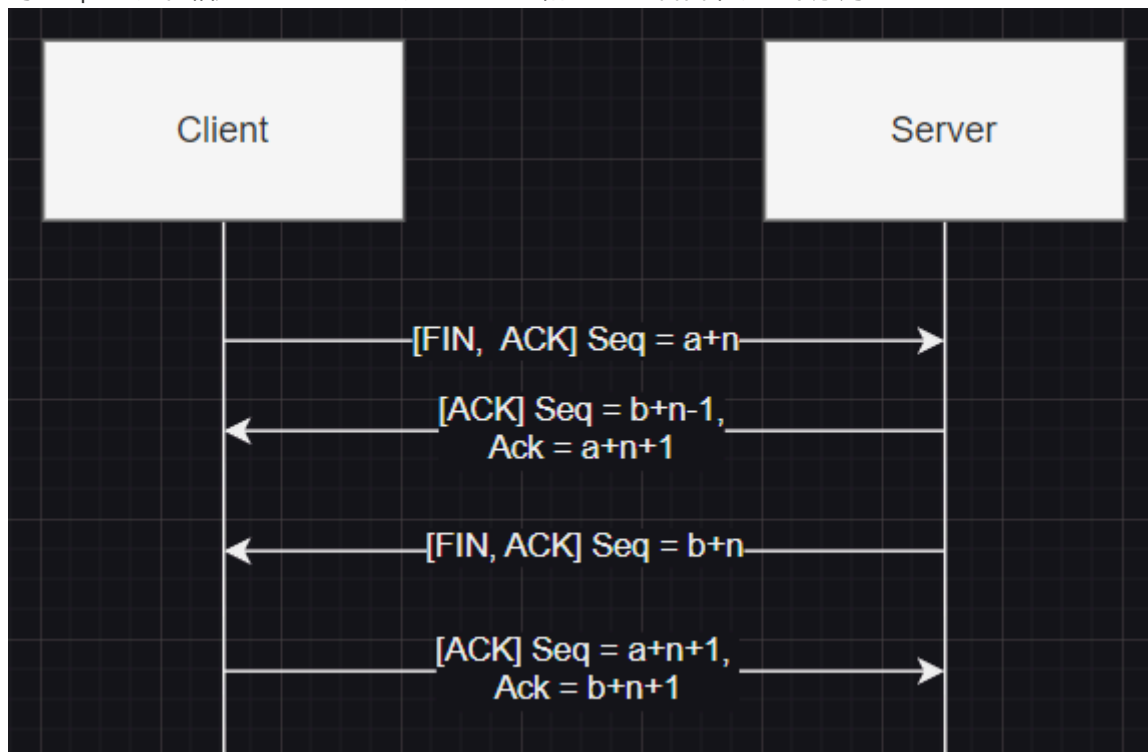
## 建立连接

连接建立我模仿 TCP 协议中的三次握手方式，Client 先向 Server 发送建立连接的请求，序列号为  $a$ ，Server 收到这条请求报文后，发送响应  $Ack = a + 1$ ，同时 Server 也开启一个序列号  $Seq = b$ ，Client 再对  $Seq = b$  进行相应  $Ack = b + 1$ 。这里 Client 和 Server 各自维护一个序列号组。



## 断开连接

断开连接我模仿 TCP 协议中的四次挥手方式，Client 先向 Server 发送断开连接的请求，序列号为  $a$ ，Server 收到这条请求报文后，发送响应  $Ack = a + 1$ ，同时 Server 也开启一个序列号  $Seq = b$ ，Client 再对  $Seq = b$  进行相应  $Ack = b + 1$ 。这里 Client 和 Server 各自维护一个序列号组。



## 差错检测

我们在报文格式设计中插入了两个函数 `check`、`setCheck` 来进行差错检测。

`check` 函数用于验证当前消息的校验和是否正确。如果传输过程中某些位被篡改，校验和会检测到不一致。

- 初始化校验和: `unsigned int sum = 0`; 初始化累加器 `sum` 为 0, 用于计算校验和。
- 数据转换为 16 位块: 使用 `unsigned short* msgStream = (unsigned short*)this`; 将整个 `Message` 结构体视为一个 `unsigned short` 的数组, 逐 16 位处理。
- 累加校验值:
  - 遍历结构体, 将每两个字节 (16 位) 累加到 `sum` 中。
  - 如果 `sum` 超过 16 位 (发生溢出), 则高 16 位与低 16 位相加, 这种处理称为“回卷”或“溢出折返”, 保证 `sum` 始终是 16 位。
- 校验结果: 最终, 如果 `sum` 的低 16 位与 `0xFFFF` 比较相等, 则校验成功, 返回 `true`。

`setCheck` 函数用于为当前消息计算校验和并设置到 `checknum` 字段, 保证消息可以通过 `check` 函数验证。

- 清零校验值: `this->checknum = 0`; 在计算前, 将 `checknum` 置为 0, 防止之前的校验值影响当前的计算。
- 数据转换为 16 位块: 和 `check` 函数类似, 将整个 `Message` 结构体视为 `unsigned short` 的数组逐 16 位处理。
- 累加校验值: 和 `check` 函数类似, 逐 16 位累加到 `sum` 中, 并处理溢出。
- 取反赋值: 将累加结果的低 16 位取反, `~(sum & 0xFFFF)`, 并赋值给 `checknum`。这一步是为了满足校验和的验证条件。

## 实验 3-1 停等协议

实验 3-1 的流量控制采用**停等机制**, 即发送方在发送一个数据包后, 必须停止并等待接收方的确认 (ACK) 信号, 只有在收到 ACK 后, 才能发送下一个数据包。

这种方式确保了每个数据包都被确认接收, 从而避免了数据的丢失或乱序。

### 连接建立的超时重传

Client 发送第一次握手, 同时开始计时, 等待 Server 发来的第二次握手。

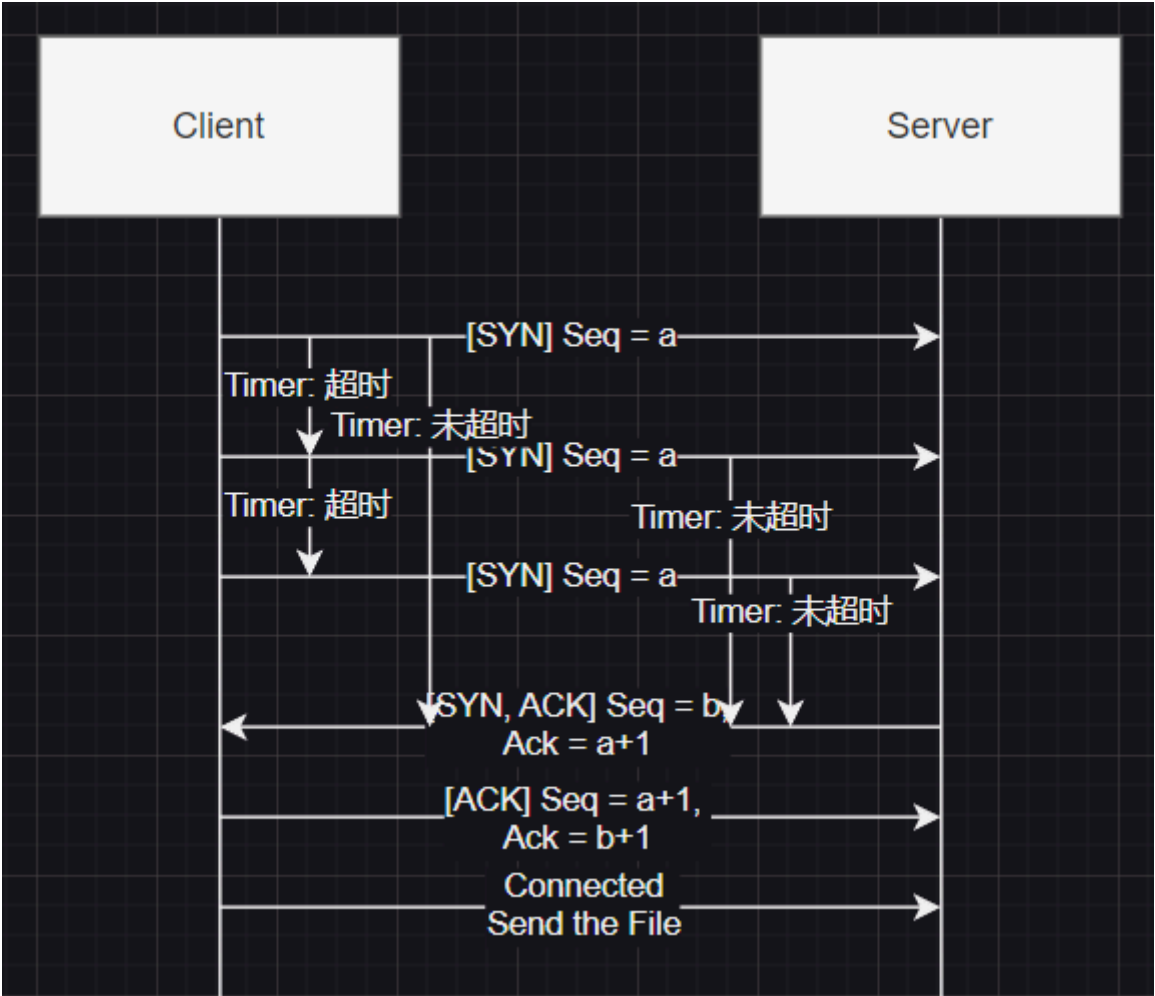
如果计时器超时 (超出在宏定义中设置的时间), Client 则会重新发送第一次握手。

如果重传次数超过在宏定义中设置的最大次数, 则默认为失败, 退出连接。

同 Client 原理一样, Server 接收到第一次握手后, 发送第二条握手, 并开始计时, 等待第三条握手。

如果计时器超时 (超出在宏定义中设置的时间), Server 则会重新发送第二次握手。若重传次数超过最大次数, 则默认失败退出连接。

连接建立超时重传示意图：



### 连接断开的超时重传

类似于建立连接的超时重传。

Client 发送第一次挥手，同时开始计时，等待 Server 发来的第二次挥手。

Client 等待第二次挥手，如果计时器超时（超出在宏定义中设置的时间），Client 则会重新发送第一次挥手。若重传次数超过最大次数，则默认失败退出连接。

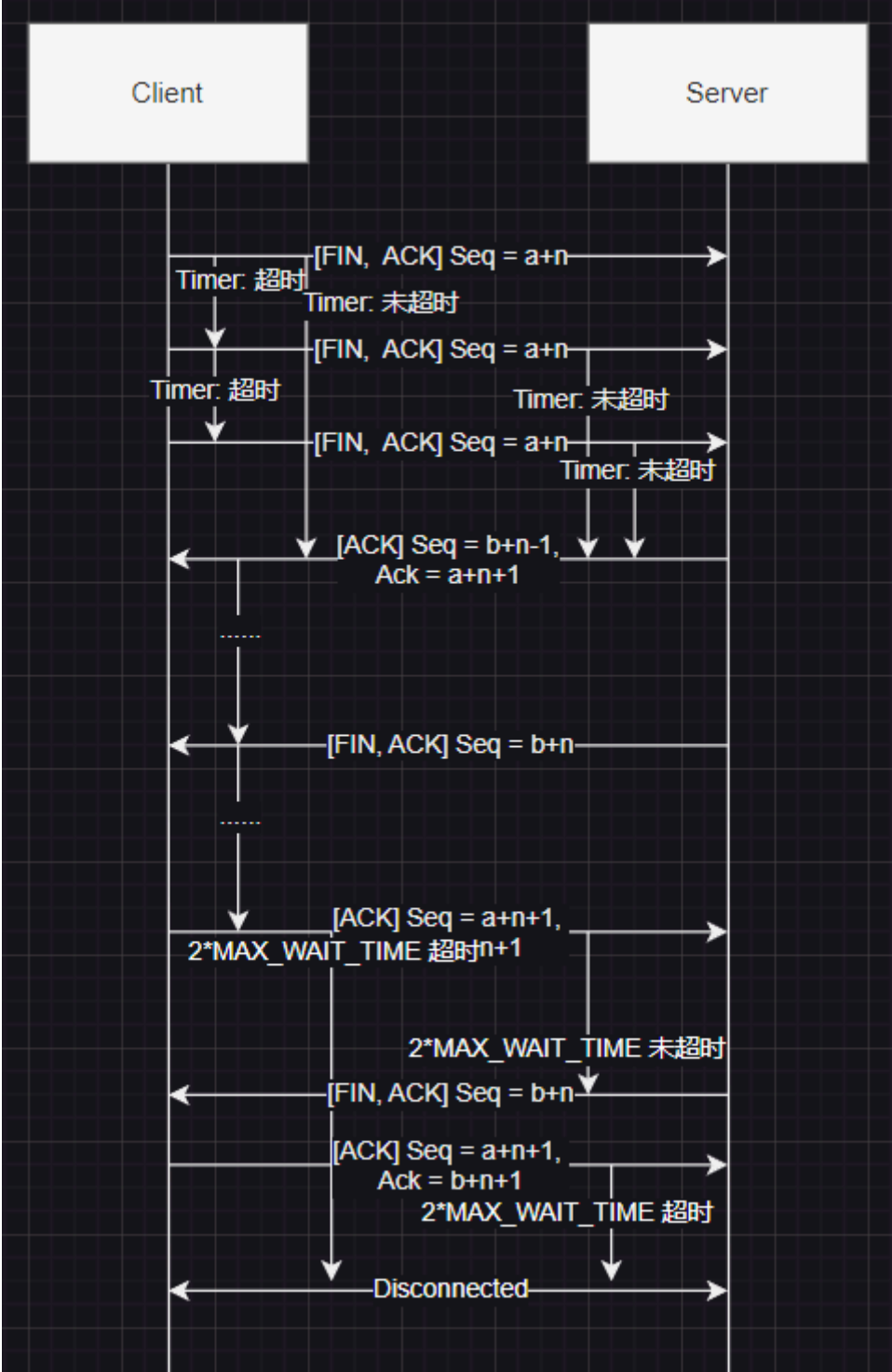
Client 等待第三次挥手，接收到第三次挥手后，发送第四次挥手。

**注意：**由于 Client 并不知道 Server 是否正确收到了第四次挥手，所以 Client 发送完第四次挥手后，需要等待大于一个最长等待时间的时间，看是否接收到 Server 再次发送的第三次挥手的消息。若接收到。证明第三次挥手丢失，需要重传。

同理，Server 接收到第一次挥手后，发送第二次挥手。检查是否超时、重传次数是否超过最大次数。

之后发送第三次挥手，开始计时并等待接收第四次挥手。检查是否超时、重传次数是否超过最大次数。

连接断开超时重传示意图：



### 文件传输的超时重传

本次实验是实现从 Client 到 Server 的单向传输，采用停等协议。四个文件均大于单次传输最大字节数，所以文件的传输是需要将文件拆分成多个报文数据包进行传输。

报文数据包需要分别装载文件名字及大小、文件内容，其中装载文件内容的数据报文包还分为满载报文和未满载报文。

正常情况无丢包时，传输过程是没有失序问题的。此时 Client 和 Server 双方只需按照顺序接收和发送。Client 发送报文数据包并开始计时，等待接收 ACK 报文，若超时未接受到报文，则重传；若接收到，则继续发送下一个报文包。

在测试过程中模拟数据包丢失情况，当 Client 发送的数据报文包丢失时，由于在发送时 Client 为每一个报文包设置了一个计时器，超时未接受到对应的 Ack 报文时，则会重传并重新计时。当重传次数超过最大次数后，则会认为和 Server 断开连接，退出程序。

当 Server 发送的 ACK 报文包丢失时，由于 Client 的超时重传，Server 会重新接收到这个重复的报文

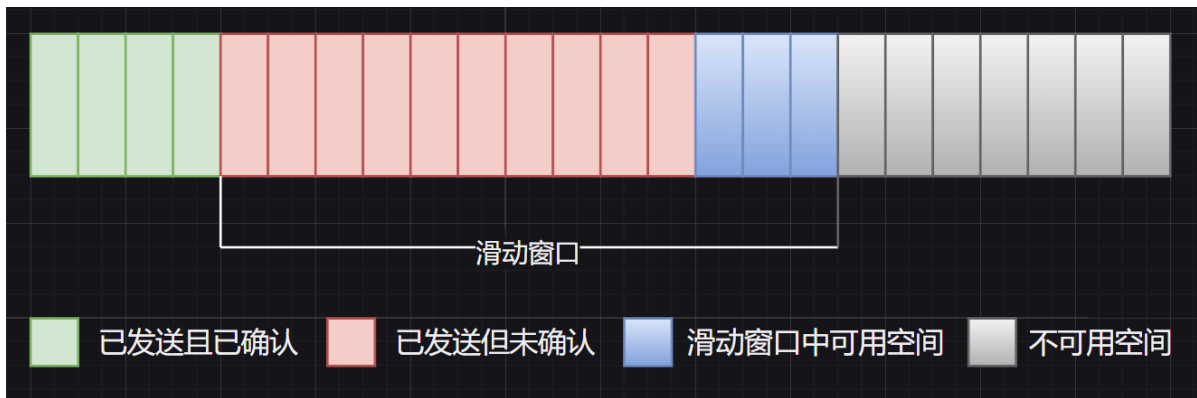
**文件传输超时重传示意图:**



根据实验要求，这里我设置发送端 Client 为 32 大小的滑动窗口，接收端 Server 窗口大小为 1。在 3-1 的实验中，我们采用停等协议，发送端 Client 发送数据后，等待接收端 Server 的确认。接收端 Server 收到数据后，发送确认信息。发送端 Client 收到确认信息后，发送下一条数据。这样明显传输效率过慢，传输时间过长。

发送端 Client 发送数据时，首先将数据放入发送窗口，然后发送窗口右移一位。发送窗口满时，发送端 Client 等待接收端 Server 的确认。接收端 Server 收到数据后，将数据放入接收窗口，然后发送确认信息。发送端 Client 收到确认信息后，将发送窗口左移一位。发送窗口有空时，发送端 Client 可以继续发送数据。

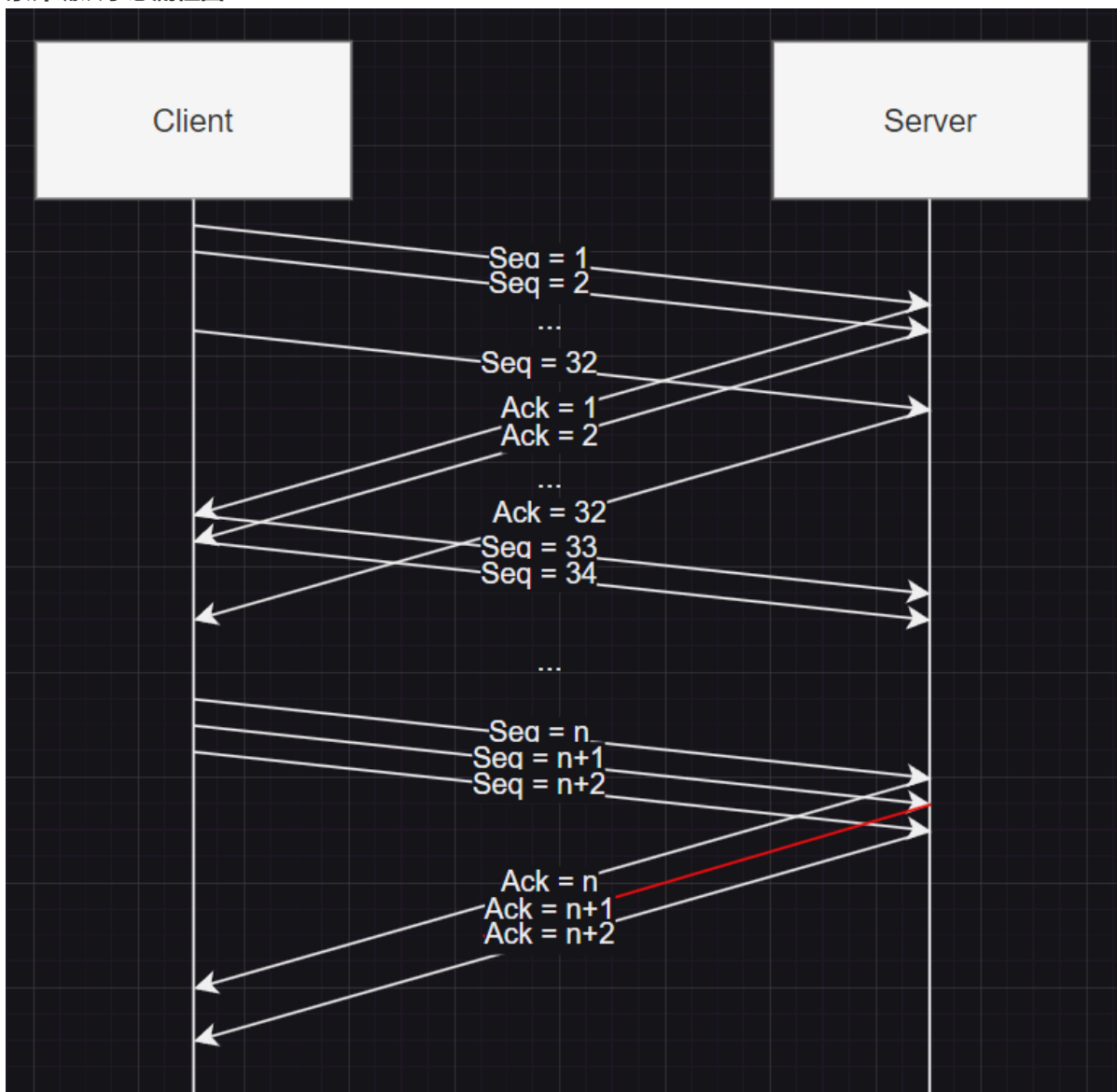
**滑动窗口示意图:**



这里我们采用 GBN 的累计确认协议。

累计确认机制主要是应对 ACK 报文丢失的问题，根据实验要求 Server 窗口大小为 1，确保接收数据报文是按序接收。累计确认保证只要 Client 收到了一个 ACK 报文，就说明前面的数据报文都被正确接收到了。如果发生了 ACK 报文丢失，Client 仍可以通过后面发送的 ACK 报文去确认之前所有的报文已成功被 Server 接收。

**累计确认示意图：**



GBN 协调称回退 N 帧协议，在连续发送数据帧时，当检测到失序或错误时，要求重传最后正确接收帧后的所有未确认帧。如果超时未收到确认，将重传最近 N 个帧。接收端只按序接受，即使接收到正确但不按序的帧也会丢弃，并重复发送确认帧以防确认丢失。

## 超时重传，快速重传

这里相对于实验 3-1，取消了最大传输次数。

**快速重传：**当 Client 丢失数据包  $n$  时，当后续数据包  $n+1$ ， $n+2$  和  $n+3$  到达时，由于累计确认机制，Server 会都返回 ACK 为  $n$  的报文。当 Client 接收到三次一样的 ACK 报文，会触发快速重传机制，将丢失的数据包  $n$  以及后面已发送的数据包重传。

**超时重传：**Client 每发送一个数据包都会为这个数据包设置一个定时器，当定时器超时后，Client 会重传这个数据包以及后面已发送的数据包。

## 实验 3-3 拥塞控制机制

### RENO 拥塞控制算法

RENO 算法有三个核心状态：

- SLOW\_START（慢启动）：连接初始状态，cwnd 从 1 开始指数增长。
- CONGESTION\_AVOIDANCE（拥塞避免）：当 cwnd 达到 ssthresh 时进入，cwnd 线性增长。
- FAST\_RECOVERY（快速恢复）：发生轻微拥塞时进入，快速调整 cwnd。

#### 1. 慢启动阶段：

- 初始状态下，cwnd 为 1。
- 每收到一个 ACK，cwnd 增加 1。
- 当 cwnd 达到 ssthresh 时，进入拥塞避免阶段。

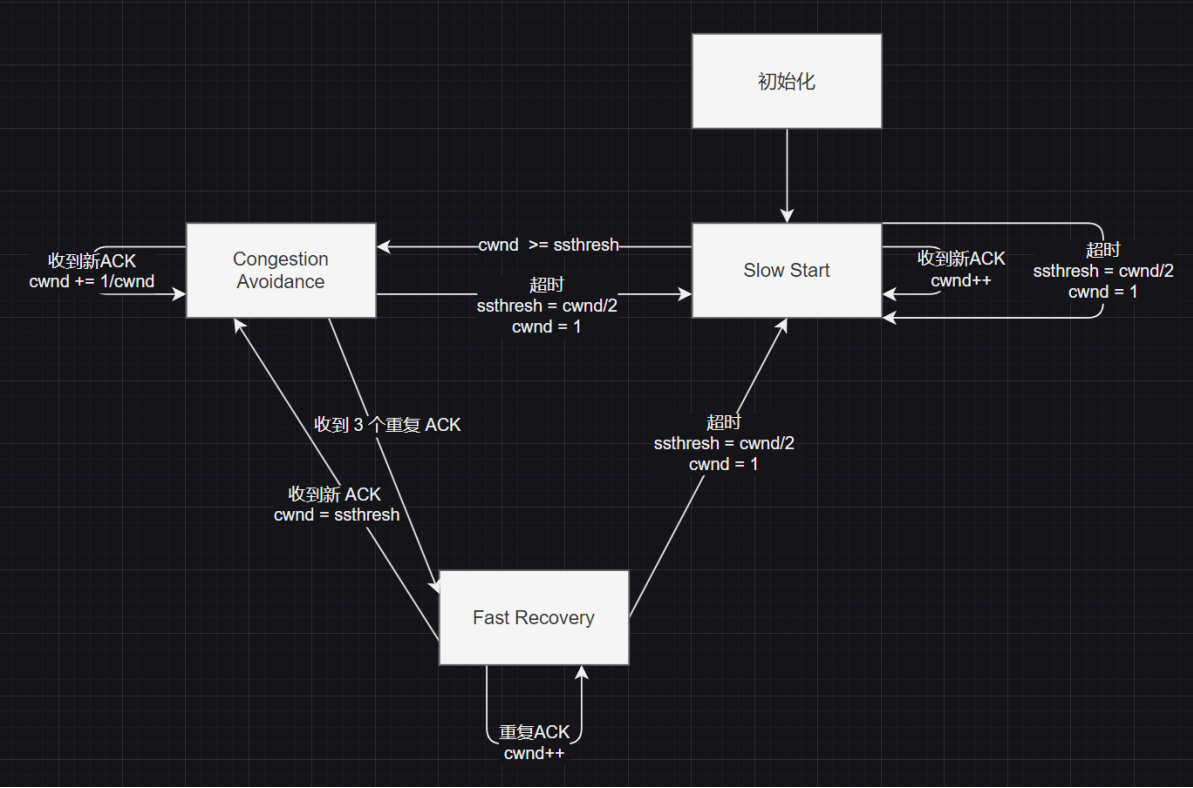
#### 2. 拥塞避免阶段：

- 每收到一个完整的 cwnd 个 ACK，cwnd 增加 1。
- 使用 cwnd\_increment 实现平滑的线性增长。
- 增长速度比慢启动慢，避免网络拥塞。

#### 3. 快速恢复阶段：

- 收到 3 个重复 ACK 时进入。
- 立即重传丢失的报文段。
- 将 ssthresh 设为 cwnd 的一半。
- cwnd 设为新的 ssthresh + 3。

RENO 流程图：



## 对比分析

为了简易，这里对比只使用测试文件中的 1.jpg。  
地址使用的是本地回环地址 127.0.0.1，发送端、路由器、接收端对应端口如下：

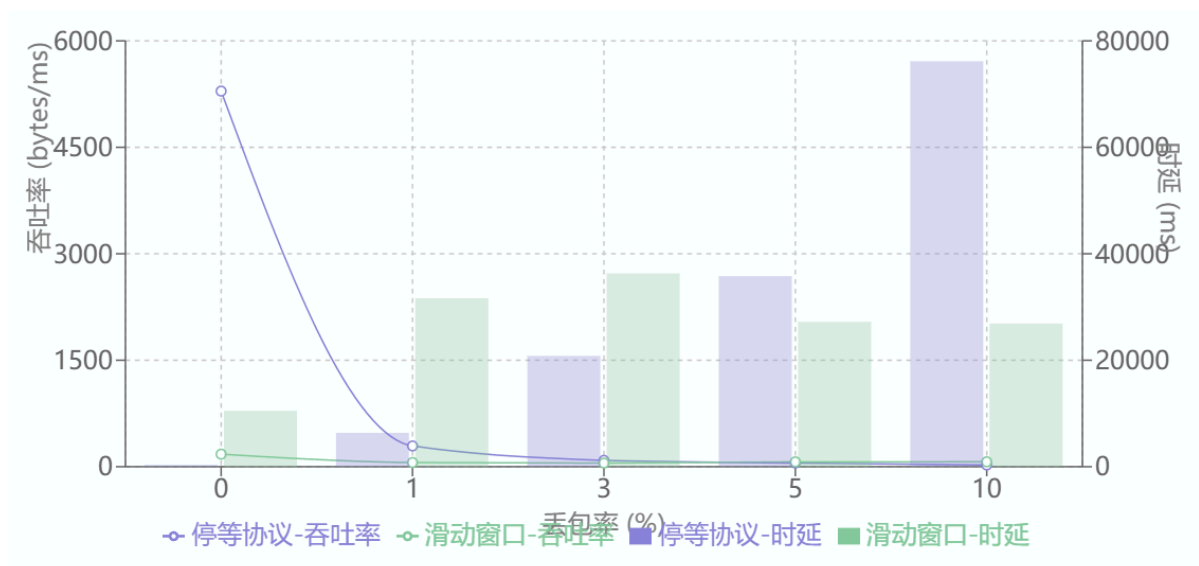
```
ClientPORT = 110
RouterPORT = 111
ServerPORT = 112
```

超时时间设置为 5000ms。  
为保证科学性，性能对比分析均采用控制变量法。同时为了避免路由器 Router 故障以及网络波动，数据采集使用 3 次数据的中间值。

## 停等机制与滑动窗口机制性能对比

### 控制丢包率

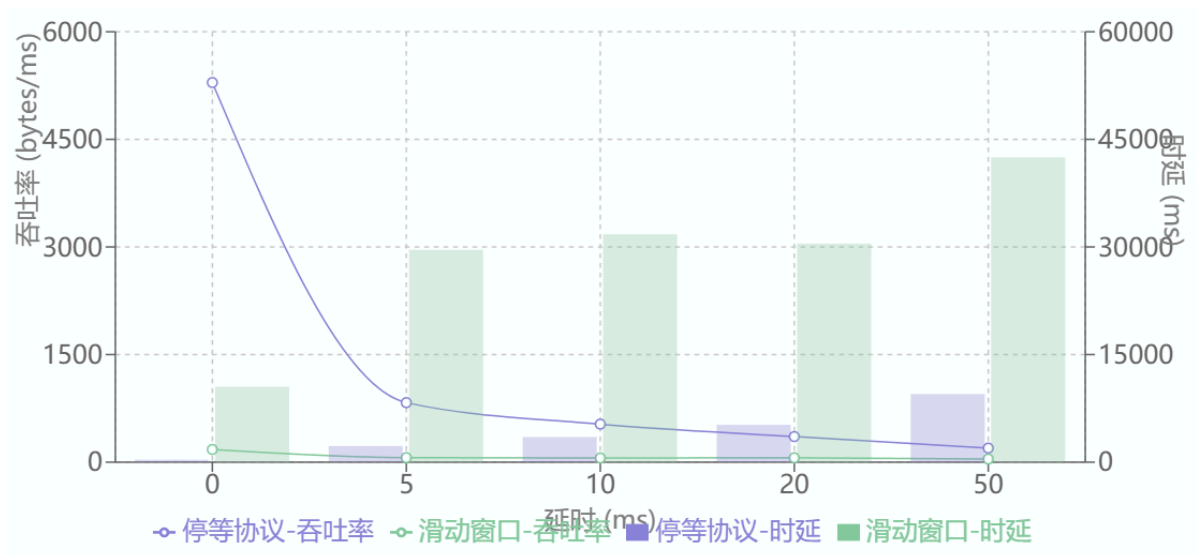
| 丢包率(%) | 停等协议   |               | 滑动窗口   |               |
|--------|--------|---------------|--------|---------------|
| 延时:0ms | 时间(ms) | 吞吐率(bytes/ms) | 时间(ms) | 吞吐率(bytes/ms) |
| 0      | 351    | 5291.6        | 10498  | 176.924       |
| 1      | 6352   | 292.404       | 31643  | 58.6971       |
| 3      | 20810  | 89.2529       | 36301  | 51.1653       |
| 5      | 35797  | 51.8857       | 27206  | 68.27         |
| 10     | 76159  | 24.3878       | 26880  | 69.098        |



根据可视化图表可以看出，停等协议的时间随着丢包率的增加而急剧增长，吞吐率随之急剧降低；滑动窗口协议所耗费的时间在低丢包率的情况下大于停等协议，在丢包率为 5% 时所耗时间小于停等协议，在 5% 之后似乎进入一个临界值。这是由于停等协议可以在低丢包率下快速传输，而滑动窗口协议在低丢包率下需要等待一段时间才能传输，所以在低丢包率下滑动窗口协议的时间比停等协议的时间要长。而大于某个值后，停等协议由于大量丢包，在等待重传的情况下所耗费的时间急剧增加，超过了滑动窗口协议的等待时间。

控制延时

| 延时(ms) | 停等协议   |               | 滑动窗口   |               |
|--------|--------|---------------|--------|---------------|
| 丢包率:0% | 时间(ms) | 吞吐率(bytes/ms) | 时间(ms) | 吞吐率(bytes/ms) |
| 0      | 351    | 5291.6        | 10498  | 176.924       |
| 5      | 2239   | 829.546       | 29593  | 62.7633       |
| 10     | 3503   | 530.218       | 31778  | 58.4478       |
| 20     | 5207   | 356.703       | 30452  | 60.9928       |
| 50     | 9491   | 195.696       | 42484  | 43.7189       |

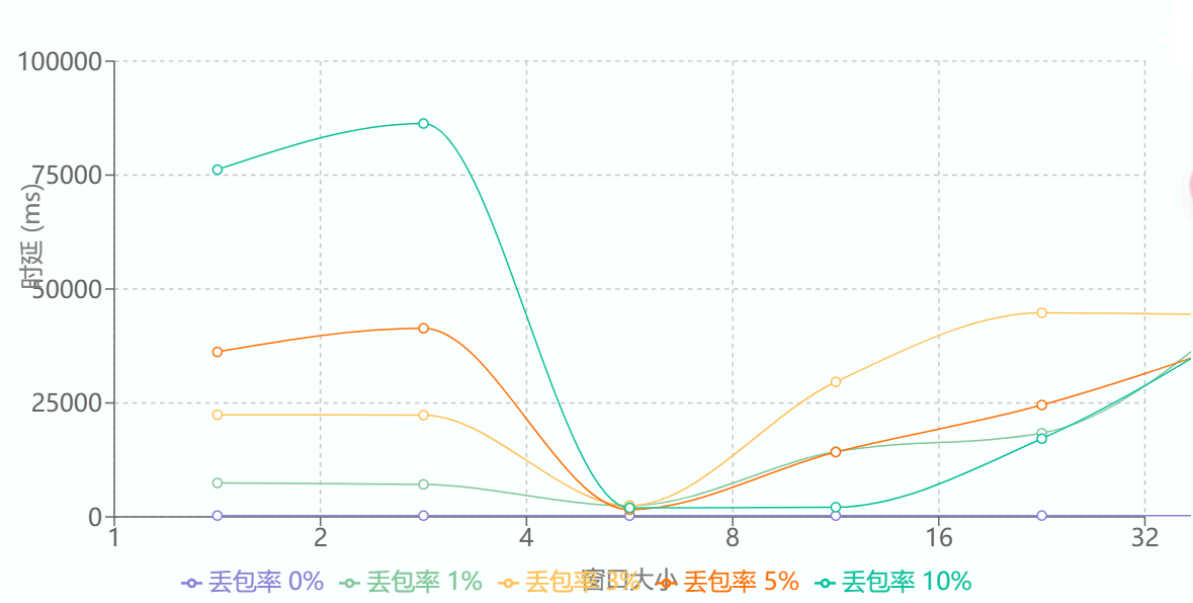
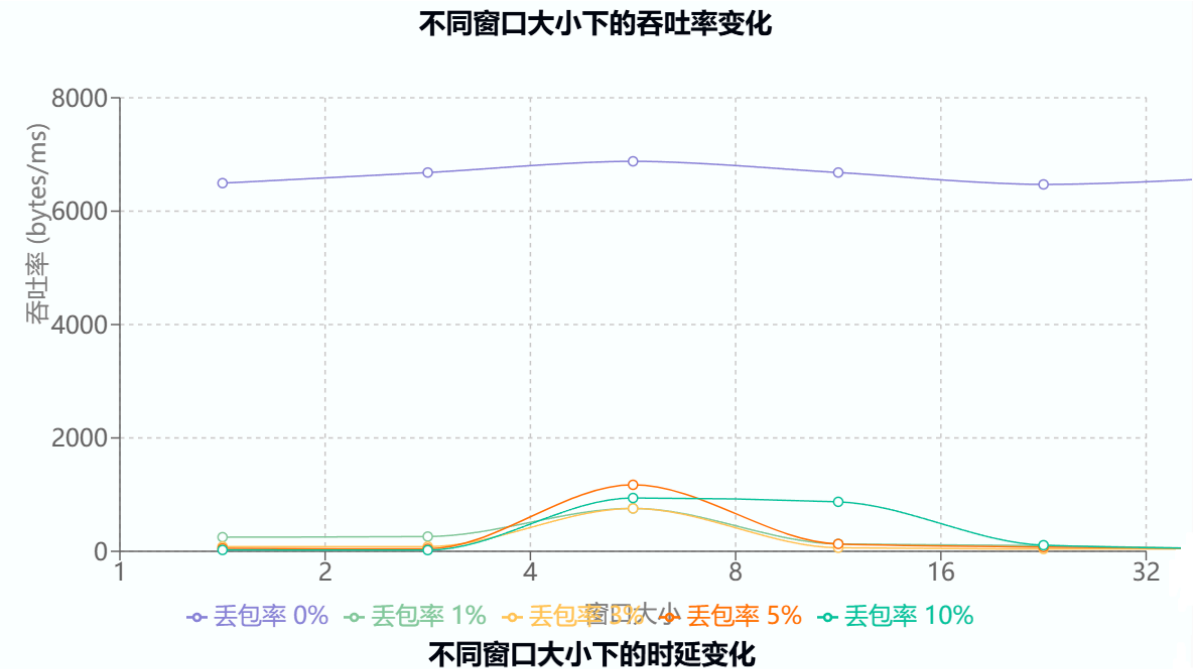


在丢包率为 0% 的情况下，停等协议无需等待丢包的延迟，所以耗费的时间整体明显低于滑动窗口协议，随着延时的增加，停等协议所耗费的时间基本成线性增加，而滑动窗口协议在延时增加时，增长函数类似于一个三次函数，在中间增长速度较慢甚至趋于一个稳定值。

## 滑动窗口机制中不同窗口大小对性能的影响

### 控制丢包率

| 窗口大小 | 丢包率(%)        | 0       | 1       | 3       | 5       | 10      |
|------|---------------|---------|---------|---------|---------|---------|
|      | 延时:0ms        |         |         |         |         |         |
| 1    | 时间(ms)        | 286     | 7436    | 22402   | 36204   | 76197   |
|      | 吞吐率(bytes/ms) | 6494.24 | 249.779 | 82.9101 | 51.3024 | 24.3757 |
| 2    | 时间(ms)        | 278     | 7125    | 22346   | 41388   | 86321   |
|      | 吞吐率(bytes/ms) | 6681.13 | 260.681 | 83.1179 | 44.8766 | 21.5168 |
| 4    | 时间(ms)        | 270     | 2452    | 2453    | 1583    | 1974    |
|      | 吞吐率(bytes/ms) | 6879.08 | 757.485 | 757.176 | 1173.31 | 940.908 |
| 8    | 时间(ms)        | 278     | 14312   | 29616   | 14223   | 2127    |
|      | 吞吐率(bytes/ms) | 6681.13 | 129.776 | 62.7145 | 130.588 | 873.227 |
| 16   | 时间(ms)        | 287     | 18364   | 44782   | 24547   | 17169   |
|      | 吞吐率(bytes/ms) | 6471.61 | 101.141 | 41.4754 | 75.6652 | 108.181 |
| 32   | 时间(ms)        | 281     | 46649   | 44263   | 39148   | 42310   |
|      | 吞吐率(bytes/ms) | 6609.8  | 39.8155 | 41.9618 | 47.4444 | 43.8987 |

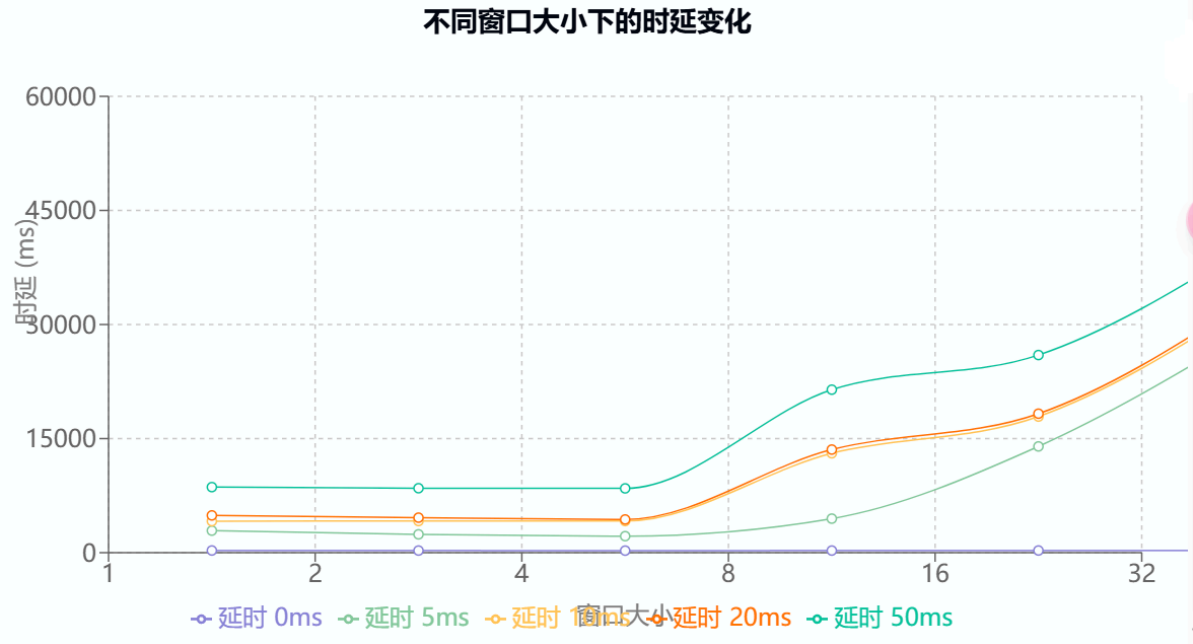
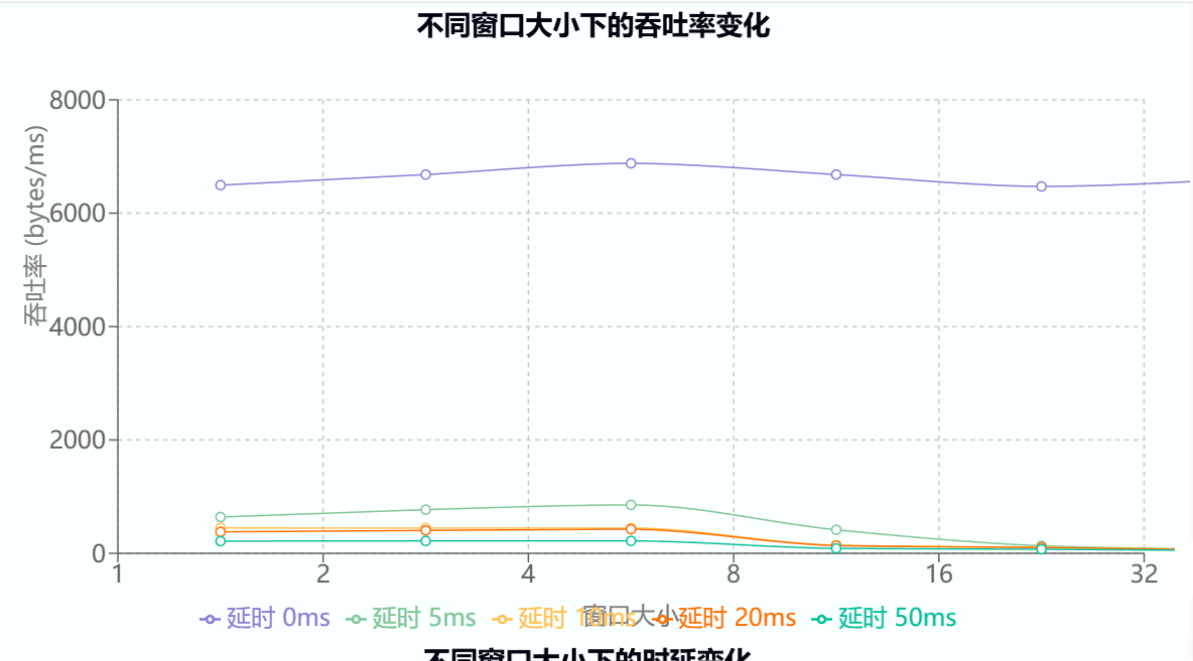


在丢包率为 0 的情况，窗口大小基本不影响时间和吞吐率。随着窗口大小的增加，时间基本呈一个增长趋势。但是在窗口大小为 4 的时候，出现了一个违背增长的情况，在窗口为 4 的情况下，所有丢包率大于 0 的曲线在窗口大小为 4 的地方出现了明显耗时减少的情况。这是可能由于在编程中所使用的数据结构在窗口为 4 时出现了吻合的情况，各部分运行时间可以完美融合，不存在进程之间相互等待的情况。同时随着丢包率的增加，通过不同窗口下的时延变化的图像，可以看出丢包率越大，在窗口大于 4 的时候，所耗时间会明显降低，体现了优化效果。

控制延时

| 窗口大小 | 延时(ms)        | 0       | 5       | 10      | 20      | 50      |
|------|---------------|---------|---------|---------|---------|---------|
|      | 丢包率:0%        |         |         |         |         |         |
| 1    | 时间(ms)        | 286     | 2900    | 4137    | 4902    | 8620    |
|      | 吞吐率(bytes/ms) | 6494.24 | 640.467 | 448.961 | 378.897 | 215.47  |
| 2    | 时间(ms)        | 278     | 2411    | 4162    | 4605    | 8476    |
|      | 吞吐率(bytes/ms) | 6681.13 | 770.366 | 446.265 | 403.334 | 219.131 |
| 4    | 时间(ms)        | 270     | 2173    | 4161    | 4366    | 8454    |

| 窗口大小 | 延时(ms)        | 0       | 5       | 10      | 20      | 50      |
|------|---------------|---------|---------|---------|---------|---------|
|      | 吞吐率(bytes/ms) | 6879.08 | 854.741 | 446.372 | 425.413 | 219.701 |
| 8    | 时间(ms)        | 278     | 4477    | 13072   | 13557   | 21433   |
|      | 吞吐率(bytes/ms) | 6681.13 | 414.866 | 142.086 | 137.003 | 86.6586 |
| 16   | 时间(ms)        | 287     | 13971   | 17894   | 18275   | 25985   |
|      | 吞吐率(bytes/ms) | 6471.61 | 132.943 | 103.798 | 101.634 | 71.4779 |
| 32   | 时间(ms)        | 281     | 28773   | 32383   | 32953   | 40089   |
|      | 吞吐率(bytes/ms) | 6609.8  | 64.5519 | 57.3558 | 56.3637 | 46.3307 |

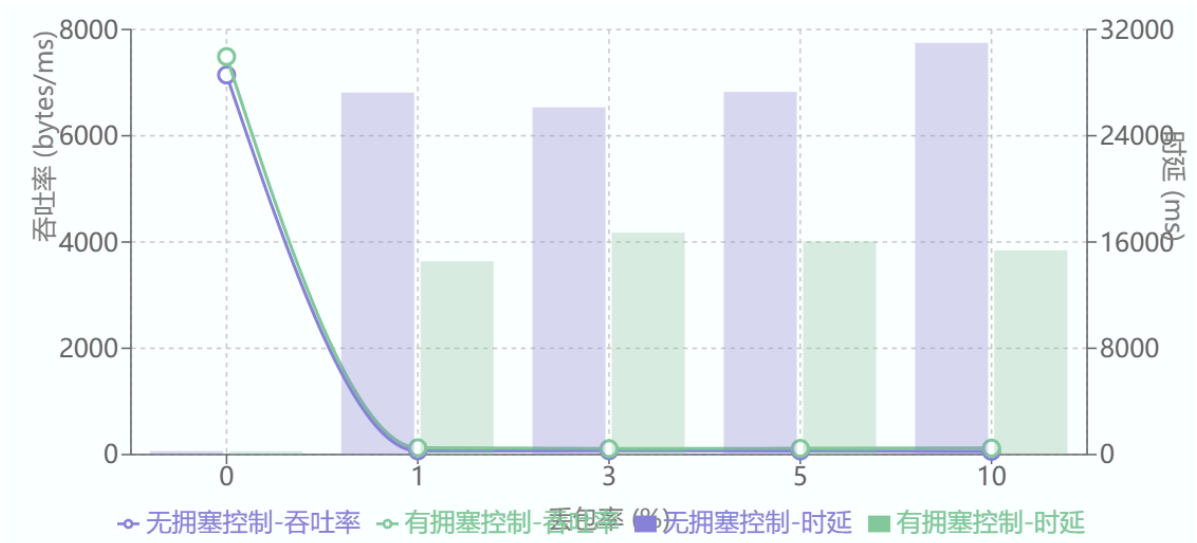


在丢包率为 0 的情况下，随着延时的增加，各个窗口大小所耗费的时间基本呈倍数增加。但在窗口大小为 4 之前，窗口大小基本不影响时间和吞吐率。大于 4 之后随窗口增大所耗费时间增加。

# 有拥塞控制和无拥塞控制的性能比较

## 控制丢包率

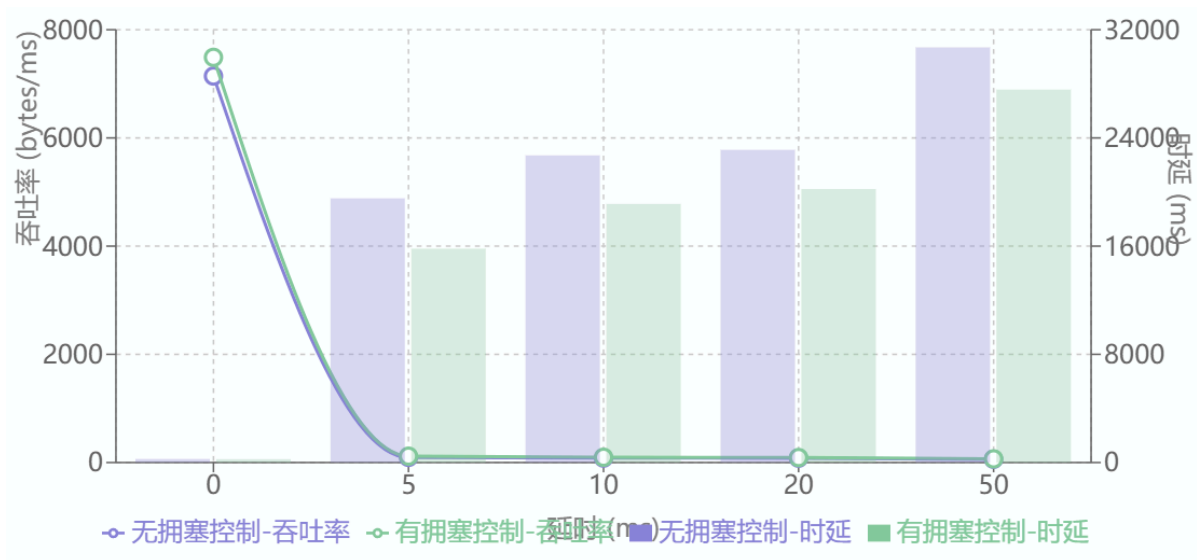
| 丢包率(%) | 无拥塞控制  |               | 有拥塞控制  |               |
|--------|--------|---------------|--------|---------------|
| 延时:0ms | 时间(ms) | 吞吐率(bytes/ms) | 时间(ms) | 吞吐率(bytes/ms) |
| 0      | 260    | 7143.67       | 248    | 7489.33       |
| 1      | 27251  | 68.1572       | 14551  | 127.644       |
| 3      | 26126  | 71.0921       | 16704  | 111.192       |
| 5      | 27298  | 68.0399       | 16047  | 115.745       |
| 10     | 30995  | 59.9243       | 15365  | 120.882       |



加上拥塞控制后，可以明显看出在添加前后的优化效果。在丢包率为 0 的情况下，有拥塞控制对于时间和吞吐率的影响不大。随着丢包率的增加，拥塞控制的优化效果明显。所耗时间基本为不加拥塞控制的耗时的一半。

## 控制延时

| 延时(ms) | 无拥塞控制  |               | 有拥塞控制  |               |
|--------|--------|---------------|--------|---------------|
| 丢包率:0% | 时间(ms) | 吞吐率(bytes/ms) | 时间(ms) | 吞吐率(bytes/ms) |
| 0      | 260    | 7143.67       | 248    | 7489.33       |
| 5      | 19534  | 95.0831       | 15808  | 117.494       |
| 10     | 22709  | 81.7893       | 19118  | 97.1521       |
| 20     | 23120  | 80.3353       | 20223  | 91.8436       |
| 50     | 30689  | 60.5218       | 27573  | 67.3613       |



在控制延时的情况下，拥塞控制也有明显的优化效果。但对比控制丢包率的情况下，优化没有那么明显。但相对于无拥塞控制的情况下，优化后所耗时间大概减少在一个定值。

## 总结反思

总体来说，实验三实现了一个传输文件的可靠传输机制，从停等协议，优化到滑动窗口加累计确认机制，再到添加拥塞控制，在这次实验中我对于 TCP 协议的机制更加了解，同时通过编程更加熟悉工作流程与机制。

在实验中，我基本实现了所需要的功能，仿照 TCP 相关协议进行设计。但是在实验 3-2 和 3-3 的过程中，在优化实现的过程中并没有起到符合理论情况的优化效果。在思考过后我认为在实验中我使用了多线程编程，但是在询问周围同学的时候，他们的多线程任务可以在 1s 内传送文件 1.jpg。所以我认为我表面似乎实现了多线程编程，但是对于程序可能只是多了负担。其次，有可能时 Router 的原因以及我的网络波动，在测试数据的过程中总出现 Router 中没有显示丢包 Miss a packet，但是程序卡在一个地方的情况。

总之，在这次实验中我学到了很多，也出现了很多我没有解决以及很迷惑的问题。相信随着我的了解学习深入我会找到答案。